WordWreck
Code Defense Guide

A practical map of the Java code, gameplay logic, and JavaFX wiring

> **The one-sentence explanation**
>
> WordWreck is a two-word crossing puzzle: the Java-only GameEngine decides tile marks, anchor survival, endings, and score; PrimaryController connects those rules to the JavaFX screen, optional dictionary checks, sound, and saved settings.

If the professor asks where the logic is

• Open **demo/src/main/java/com/example/preview/GameEngine.java**. Start at **submitGuess** (lines 73-112), then **evaluateLetters** (lines 118-146), then **scorePercent** (lines 60-67).

• For the anchor on the screen, open **PrimaryController.java** at lines 219-236 and 311-347. For online approval, go to lines 348-410.

• The playable answers come from **Levels.ALL**, not the old **backend** package. The UI uses those curated levels directly.

Your quickest practice order

1. Explain Level 1 (CARGO / SHARKS) and the shared R. 2. Trace one Enter press through controller to engine. 3. Walk through the two-pass color algorithm. 4. Explain all five endings and scores. 5. Explain why a late dictionary result cannot approve newly typed letters.

`Source: demo/src/main/java/com/example/preview/Levels.java:3-25`

Reading this guide

Paths are relative to **C:\School\WordWreck**. A reference such as **GameEngine.java:73-112** means those source lines. The guide reflects the current newer copy with sounds and graphics. The older friend's folder did not supply missing live game features.

1. The actual program architecture

Know which file owns each decision.

Layer	What it does	Where to open
Entry and window	Starts JavaFX, loads the font, FXML and CSS, creates the Stage/Scene, scales the screen.	App.java:20-57, 82
Screen controller	Receives input, maintains the typed letters, calls the engine, paints UI, manages lookup and sounds.	PrimaryController.java:34-65, 67-116
Live game rules	Stores the current level, guesses, anchor, solved states, outcome and score.	preview/GameEngine.java:30-67, 73-146
Level data	Defines two crossing answers, clue strings and zero-based anchor positions; has 20 fixed levels.	preview/Level.java:3-20; Levels.java:3-25
Word approval	Optional online dictionary check, in-memory result cache and response classification.	DictionaryService.java:14-69
Local settings	Saves mode, motion, audio choices and per-level best scores via Java Preferences.	GameSettings.java:7-47
Presentation helpers	Defines hints, pixel ocean drawing, sound effects and looping music.	PirateHints.java:8-52; OceanArt.java:7-153; SoundManager.java:15-152

> **Do not claim the old backend runs the game**
>
> The `backend` package is not called by App or PrimaryController. TableLogic is empty; BackendLauncher independently prints a word from words.txt; WordScraper independently reads example.txt. The playable model is GameEngine plus Levels.ALL.

The dependency direction is simple: **App loads FXML/controller** -> **controller creates GameEngine(Levels.ALL.get(index))** -> **controller calls submitGuess** -> **engine returns state/marks** -> **controller redraws the screen**.

`Source: demo/src/main/java/com/example/PrimaryController.java:3-8, 197-209, 243-270`

Why the folders stay as they are

Java source belongs under **demo/src/main/java**; FXML, CSS, font, sound and words belong under **demo/src/main/resources**. App and SoundManager load those resources from the classpath. Maven's pom.xml declares JavaFX controls, FXML and media. Moving everything into one `codes` folder would require rewriting the build and resource paths, so the current structure is the simplest safe runnable layout.

`Source: demo/pom.xml:1-19`

2. Game rules, anchor, and outcomes

Use Level 1 for every live explanation.

Level 1 is **CARGO** across and **SHARKS** down. The Level constructor receives `anchorA=2` and `anchorB=3`. Java indexes start at 0: CARGO[2] and SHARKS[3] are both **R**. The constructor checks that the crossing letters match and the positions exist. The screen labels those as A letter 3 and B letter 4 because it adds 1 for people.

`Source: demo/src/main/java/com/example/preview/Levels.java:6`

`Source: demo/src/main/java/com/example/preview/Level.java:12-20`

`Source: demo/src/main/java/com/example/PrimaryController.java:311-345`

The anchor rule

A correct **green** letter at A's exact anchor position secures the crossing. `anchorSecured |= ...` means once secured, it stays true for the rest of that level. A yellow R elsewhere does not secure it. If A reaches five misses with the anchor secured, B still opens. If A reaches five misses without it, the level ends immediately. Once B opens, its shared R is prefilled and cannot be changed or erased.

`Source: demo/src/main/java/com/example/preview/GameEngine.java:93-108`

`Source: demo/src/main/java/com/example/PrimaryController.java:219-236`

Five possible final outcomes

Outcome and score	How it happens	Level 1 demo (offline practice)
FULL_VICTORY - 100%	Solve A and B.	A=CARGO, then B=SHARKS.
PARTIAL_SURVIVAL - 50%	Solve A; miss B five times.	A=CARGO; B=AAARAA five times.
SURVIVAL_CLEAR - 50%	Miss A five times with green anchor; solve B.	A=AARAA five times; B=SHARKS.
BOTH_SUNK - 0%	Miss A with anchor; miss B five times.	A=AARAA five times; B=AAARAA five times.
IMMEDIATE_SINK - 0%	Miss A five times without green anchor.	A=XXXXX five times; B never opens.

Fake guesses in this table are accepted only in **Offline Practice**. With dictionary mode ON, those guesses may fail word approval before the engine is called. Repeated guesses still consume attempts. A correct answer on guess 5 wins because the correct-answer branch is checked before the five-miss branch.

`Source: demo/src/main/java/com/example/preview/GameEngine.java:73-112`

3. Walk through submitGuess line by line

This is the most important function for a logic defense.

Lines	What happens	Why it matters
73-76	`submitGuess(String input)` refuses guesses after the outcome ends.	A finished level cannot gain another attempt.
78-82	Null becomes empty; text is trimmed and uppercased; `[A-Z]{length}` checks exact current answer length.	Only complete alphabetic guesses enter the engine.
83-86	On B, check the guess at `anchorB` against B's fixed crossing letter.	The anchor stays locked even if engine is called directly.
88-91	Calculate marks; create a Guess; append it to A or B history; test exact whole-word equality.	A submitted guess spends one attempt and stores its feedback.
93-95	If still on A, OR the green anchor result into `anchorSecured`.	Earlier green anchor cannot be lost on later guesses.
96-102	If A is solved, switch to B; else on A's fifth guess, switch only if anchor held, otherwise immediate sink.	This is the A survival fork.
103-109	If B is solved, choose FULL_VICTORY or SURVIVAL_CLEAR from A state; else on B's fifth miss, choose PARTIAL_SURVIVAL or BOTH_SUNK.	This is the B ending fork.
111	Return the Guess with its marks.	The caller can inspect feedback; the engine state is already updated.

> **How to explain the parameters**
>
> `input` is the proposed word, passed by the controller as `new String(typedLetters)`. `currentAnswer()` picks A or B from `workingOnWordB`. In `evaluateLetters(answer, guessedWord)`, `answer` is the hidden target and `guessedWord` is what the player submitted. `wordB` and `down` booleans use false for A/across, true for B/down.

The engine does **not** contact the dictionary. The controller has already checked whether the current string can be submitted. This separation lets the same rules work in both online and offline modes.

`Source: demo/src/main/java/com/example/preview/GameEngine.java:43-67, 69-112`

`Source: demo/src/main/java/com/example/PrimaryController.java:243-252`

4. How tile colors are calculated

The two passes prevent duplicate letters from earning too many yellow tiles.

`evaluateLetters(answer, guessedWord)` returns one Mark per position: **CORRECT** means same letter and same spot (green); **PRESENT** means the letter exists in another unused spot (gold); **ABSENT** means no remaining copy matches (gray). The result is a list ordered exactly like the guess.

`Source: demo/src/main/java/com/example/preview/GameEngine.java:13, 114-146`

Step	Code lines	Meaning
1. Length	119-121	Reject a direct call whose answer and guess lengths differ.
2. Default	123-125	Start every tile as ABSENT and make a map for unused answer-letter counts.
3. Green pass	127-134	For each position: exact match becomes CORRECT. Otherwise count the answer's letter at that position.
4. Yellow pass	136-144	For each non-green guess letter: if its remaining count is positive, mark PRESENT and subtract one.
5. Result	145	Return an unmodifiable copy of the marks.

Work a repeated-letter example aloud

Answer **CARGO**; guess **RACER**. In the green pass only A at position 1 is exact. The remaining unmatched answer letters are C:1, R:1, G:1, O:1. In the second pass, the first R uses up R:1 and becomes gold; C uses C:1 and becomes gold. E has no count, and the final R has no R left, so both are gray.

Index	0	1	2	3	4
Answer	C	A	R	G	O
Guess	R	A	C	E	R
Mark	PRESENT	CORRECT	PRESENT	ABSENT	ABSENT

Another quick example: CARGO vs AAAAA gives [ABSENT, CORRECT, ABSENT, ABSENT, ABSENT]. One A in the answer cannot color five tiles. The test file explicitly checks both examples.

`Source: demo/src/test/java/com/example/preview/GameEngineTest.java:32-33`

5. From a key press to a result

Trace what happens in PrimaryController.

Stage	Exact route
Begin level	`startGame` -> `beginNewLevel(index)` chooses Levels.ALL, creates GameEngine, clears typed letters, starts sound, refreshes display. PrimaryController.java:185-209.
Type or erase	Physical KEY_TYPED/KEY_PRESSED filters and on-screen buttons call `typeLetterIntoFirstEmptyTile`, `eraseLastTypedLetter`, or `submitGuess`. Lines 95-112, 462-479.
Typed slots	`typedLetters` is a char[]; numeric value 0 means an empty tile. On B, `resetTypedLettersForCurrentWord` inserts the anchor and `isLockedAnchorPosition` protects it. Lines 219-237.
Enter gate	`submitGuess` requires all slots full, a VALID result, and the approved word equal to the current char[] string. Lines 243-248.
Rule call	Controller calls `game.submitGuess(...)`; the engine spends the attempt and changes state. Controller resets input for next active word, then redraws or shows final screen. Lines 249-271.
Redraw	`refreshGameDisplay` updates labels, clue, board, histories, keyboard and shark threat. Lines 284-304.
Finish	`showFinalResult` chooses win/lose sound, saves best score, translates enum outcome into text and reveals both answers. Lines 495-518.

Where the crossing shape comes from

`drawCrosswordBoard` loops over rows of B and columns of A. A tile exists if its row equals B's anchor index (the across row) or its column equals A's anchor index (the down column). Both true means the shared anchor tile. The active word's typed letters temporarily cover the display. This is drawing logic, not scoring logic.

`Source: demo/src/main/java/com/example/PrimaryController.java:311-347`

Where the tile and keyboard feedback appears

`drawGuessHistoryForWord(false)` lays A guesses across in rows; `true` lays B letters downward, with each attempt as a column. `refreshKeyboardColors` gives each key its strongest observed mark. The marks themselves came from GameEngine, not the CSS.

`Source: demo/src/main/java/com/example/PrimaryController.java:427-493`

6. Dictionary modes and late responses

This is validation before a guess, not the game score calculation.

Offline Practice

After every tile is filled, the controller calls `dictionary.validateWord(word, false).join()`. This returns an already completed VALID result for a 5-9 letter A-Z string; it makes **no HTTP request**. A nonsensical complete word can therefore be submitted and counted as a wrong guess. The engine still checks target length and the B anchor.

`Source: demo/src/main/java/com/example/PrimaryController.java:361-379`

`Source: demo/src/main/java/com/example/DictionaryService.java:37-45`

Online Dictionary

The service first checks its in-memory cache. All 20 levels' true answers are pre-approved there, so the answers remain playable even if the dictionary omits a plural or is unreachable. For another word, it sends an asynchronous HTTP GET. A 404 means INVALID; a plausible dictionary JSON response with status 200 means VALID; timeout, throttling, server error or unexpected response means UNAVAILABLE and can be retried. This is a simple response-shape check, not a full JSON parser.

`Source: demo/src/main/java/com/example/DictionaryService.java:24-30, 38-69`

How an old response is prevented from unlocking Enter

Line range	Protection
348-356	Every edit or mode change stops the 400 ms wait, cancels any pending future, increases `lookupRequestVersion`, clears VALID and clears `approvedTypedWord`.
380-385	The controller captures the current version, waits 400 ms after typing, then starts the asynchronous check.
385-399	When the response arrives, `Platform.runLater` returns to the JavaFX UI thread. The callback ignores a result if version changed, the game screen is no longer active, or the typed word changed.
407-410	The Enter button only enables if the VALID result matches the exact current string. Physical Enter uses the same check in `submitGuess`.

> **Typical follow-up**
>
> If the professor asks whether an unavailable dictionary spends an attempt: no. `submitGuess` returns before calling GameEngine unless the exact current word is approved. The Retry button revalidates the word. The game does not silently switch to offline mode.

`Source: demo/src/main/java/com/example/PrimaryController.java:243-252, 348-410`

7. JavaFX for Java/C/C++ students

Think of JavaFX as the window, event, and drawing toolkit around the Java rules.

JavaFX term	Meaning in this project	Locate it
Application	Framework base class. `launch(args)` creates the runtime and invokes `start(Stage)`.	App.java:12, 20, 82
Stage	The desktop window. App sets title, size, Scene and custom maximize behavior.	App.java:20-45, 68-81
Scene	The tree of visible controls and layout panes. App installs a root page into it.	App.java:34-42
FXML	XML description of the screen. `fx:id` links a node to an `@FXML` field; `onAction='#method'` links a button to a controller method.	primary.fxml:4-86; PrimaryController.java:35-44
Controller	The Java object that receives UI events and updates screen nodes. `initialize` runs after FXML fields are injected.	PrimaryController.java:67-116
Layout panes	VBox stacks vertically, HBox horizontally, GridPane positions crossing tiles by row/column.	primary.fxml:24-86; PrimaryController.java:311-347
CSS style class	Controls colors, font and appearance. Controller assigns classes such as correct/present/absent.	wordwreck.css; PrimaryController.java:281-283
Canvas	A surface drawn directly through GraphicsContext. OceanArt paints sea, shark and island sprites.	OceanArt.java:41-153
Timeline	Runs KeyFrames to move sharks and animate sea/guess feedback.	PrimaryController.java:578-633
runLater	Schedules changes to controls on the JavaFX UI thread after an asynchronous network result.	PrimaryController.java:385-399

`showOnlyScreen` uses both `visible` and `managed`. Invisible hides a pane; unmanaged also removes its space from layout. Property listeners redraw art and threat tracks when size or animated values change. `event.consume()` prevents a handled keystroke from also reaching another control.

`Source: demo/src/main/java/com/example/PrimaryController.java:74-112, 210-217`

8. Scores, hints, sounds, and graphics

These files support the game but do not decide the word outcome.

Saved settings and score

`GameSettings` reads/writes Java Preferences on this computer. Defaults are dictionary OFF, motion ON, effects ON and music ON. `recordBestScoreForLevel` stores `max(oldBest, newScore)` so a weaker replay does not erase a stronger one. Score keys separate `practice.` from `dictionary.`. If offline mode is used at any point during a round, `usedOfflinePracticeThisRound` remains true and the final result is stored as practice. Active guesses exist only in memory; Home -> Resume works only until the app closes.

`Source: demo/src/main/java/com/example/GameSettings.java:10-47`

`Source: demo/src/main/java/com/example/PrimaryController.java:117-129, 161-173, 197-209, 495-518`

Free hints

`showHint` calls `PirateHints.forWord(game.currentAnswer())`. The hints are a fixed map of pirate-style definitions. Showing one never calls `GameEngine.submitGuess`, so it uses no attempt and changes no score.

`Source: demo/src/main/java/com/example/PrimaryController.java:412-417`

`Source: demo/src/main/java/com/example/PirateHints.java:8-52`

Audio

`SoundManager` maps sound-effect names to bundled files and caches `AudioClip`s; `MediaPlayer` loops menu and game music. Settings switch effects and music independently. The controller chooses the moment to play each sound (type, erase, start, wrong/correct, finish). Playback failures are caught so an audio problem does not crash the game.

`Source: demo/src/main/java/com/example/SoundManager.java:16-38, 49-148`

`Source: demo/src/main/java/com/example/PrimaryController.java:67-68, 117-129, 243-270, 495-502`

Art and motion

`OceanArt` contains the primitive pixel-art drawing routines. PrimaryController draws the home and game canvases, starts the sea Timeline, animates shark positions based on wrong-guess counts, and turns all motion off when the motion setting is disabled. These effects read engine state; they do not change it.

`Source: demo/src/main/java/com/example/OceanArt.java:41-153`

`Source: demo/src/main/java/com/example/PrimaryController.java:533-633`

9. Fast function locator

Use Ctrl+F on the exact name during the defense.

Question	Search for	Source line
Where are the playable words?	Levels.ALL	preview/Levels.java:3-25
Where is the anchor validated?	Invalid intersection	preview/Level.java:12-20
Where is a guess processed?	submitGuess(String input)	preview/GameEngine.java:73-112
Where is the sticky anchor?	anchorSecured |=	preview/GameEngine.java:93-101
Where are colors calculated?	evaluateLetters	preview/GameEngine.java:118-146
Where are points calculated?	scorePercent	preview/GameEngine.java:60-67
Where are misses counted?	wrongGuessCountForWord	preview/GameEngine.java:55-58
Where is a new level made?	beginNewLevel	PrimaryController.java:197-209
Where is B's R locked?	isLockedAnchorPosition	PrimaryController.java:219-236
Where is physical Enter handled?	KEY_PRESSED	PrimaryController.java:104-110
Where is on-screen Enter made?	buildOnScreenKeyboard	PrimaryController.java:462-474
Where is the Enter gate?	updateEnterButtonState	PrimaryController.java:243-248, 407-410
Where are late lookups ignored?	lookupRequestVersion	PrimaryController.java:348-399
Where is the result displayed?	showFinalResult	PrimaryController.java:495-518
Where are best scores saved?	recordBestScoreForLevel	GameSettings.java:37-46

All entries above are under **demo/src/main/java/com/example/**; `preview/` means its preview subfolder. The old `backend/` folder is a separate sibling of `com/example/` and does not feed the live screen.

10. Professor questions: core logic

Short answers you can give while pointing to code.

What is the main calculation?

`GameEngine.evaluateLetters` computes each tile mark; `submitGuess` updates game state; `scorePercent` converts the final Outcome to 100, 50 or 0. Point to GameEngine.java:60-146.

What is the anchor?

The shared crossing letter of A and B. At level 1 it is R: A index 2 and B index 3. A green match at A index 2 secures it. Point to Levels.java:6 and GameEngine.java:93-101.

Why use `|=` for the anchor?

It is a sticky OR. Once any A guess has a green crossing tile, later non-green guesses must not unset that progress. Point to GameEngine.java:95.

What if A fails all five?

With secured anchor, B opens. Without it, IMMEDIATE_SINK. Point to GameEngine.java:99-102.

What are the five endings?

Both solved: FULL_VICTORY. A solved, B failed: PARTIAL_SURVIVAL. A failed with anchor, B solved: SURVIVAL_CLEAR. Both failed after anchor: BOTH_SUNK. A failed without anchor: IMMEDIATE_SINK. Point to GameEngine.java:93-109.

How are scores calculated?

By final outcome, not number of guesses: 100 for full victory; 50 for either one-word survival; 0 for both losses. Point to GameEngine.java:60-67.

Why is a solved fifth guess a win?

The code checks `guessedCorrectAnswer` before the fifth-failure condition. Point to GameEngine.java:96-108.

Why two color passes?

Green matches reserve their exact positions first; the second pass spends only leftover letter counts for yellow. This avoids marking too many repeated letters. Point to GameEngine.java:127-145.

11. Professor questions: parameters and JavaFX

Be precise about what is passed and what each file owns.

What does `boolean wordB` or `boolean down` mean?

`false` selects A/across; `true` selects B/down. It chooses history, solved state, wrong count, status or vertical drawing. GameEngine.java:46, 50-58; PrimaryController.java:305-309, 427-460.

What is passed into `submitGuess`?

A String built from the controller's `typedLetters` array. The engine normalizes it, validates its length/anchor, scores it and stores it. PrimaryController.java:249-253; GameEngine.java:73-91.

What is passed to `evaluateLetters`?

The hidden answer and the submitted word, in that order. It returns an ordered List of marks, one per guess position. GameEngine.java:88, 118-146.

Why add 1 to an anchor index in a message?

The array positions are zero-based for Java; players count letters from 1. GameEngine.java:83-85; PrimaryController.java:344.

Can a duplicate guess be entered?

Yes. There is no history-duplicate check. Each approved submit appends a Guess and spends an attempt. GameEngine.java:88-90.

What is the role of FXML and CSS?

FXML declares screen nodes and button-handler links; CSS paints them. PrimaryController owns behavior. primary.fxml:4-86; PrimaryController.java:35-44.

What is `@FXML initialize`?

FXMLLoader creates the controller, injects matching `fx:id` fields, then calls `initialize` to wire listeners and keyboard handling. App.java:48-54; PrimaryController.java:67-116.

Why is `Platform.runLater` needed?

The dictionary future can complete off the JavaFX application thread; `runLater` updates labels and button state safely on the UI thread. PrimaryController.java:383-399.

Why keep `lookupRequestVersion`?

A response for older letters may arrive after the user edits. The version and word checks discard it so only the current word unlocks Enter. PrimaryController.java:348-399.

Does the old backend choose levels?

No. `beginNewLevel` reads `Levels.ALL`. The old backend classes are separate and never called by the screen. PrimaryController.java:197-200; Levels.java:3-25.

12. Two-minute live trace

Practice this in the editor with Level 1 open.

Time	Say and show
0:00-0:20	Open Levels.java:6. `CARGO` is A, `SHARKS` is B, and `2,3` are zero-based positions of the same R. Then open Level.java:12-20 to show the intersection check.
0:20-0:40	Open PrimaryController.java:197-209. Selecting a level constructs `new GameEngine(Levels.ALL.get(index))`, resets input and draws the game.
0:40-1:00	Open PrimaryController.java:95-112, 243-252. Physical/on-screen Enter reaches the same controller gate; after approval it passes the typed String to `game.submitGuess`.
1:00-1:25	Open GameEngine.java:73-112. A guess is normalized, length/anchor checked, evaluated, stored, then the A or B branch chooses the next state. Point to sticky anchor and the five-ending fork.
1:25-1:45	Open GameEngine.java:118-146. Explain green first, remaining counts second; use CARGO/RACER to show the duplicate R becomes gray at the end.
1:45-2:00	Open GameEngine.java:60-67 and PrimaryController.java:495-518. Outcome maps to score; the controller shows the result and saves the best score.

Quick terms and parameter card

Term	Meaning
`index` / `levelIndex`	Zero-based position in Levels.ALL. `index=0` is Level 1; display uses `levelIndex+1`.
`anchorA`, `anchorB`	Zero-based positions of the same letter in the across and down answers.
`input` / `guessedWord`	What the player is trying to submit; the engine uppercases and checks it.
`answer`	The current hidden target returned by `currentAnswer()`.
`wordB`, `down`	False means A/across; true means B/down.
`typedLetters`	Current editable char[] on the screen; a zero char means an empty tile.
`Mark` / `Outcome`	Mark is one letter's feedback; Outcome is the whole level's state/end result.

> **Last rehearsal tip**
>
> When asked about a function, state (1) what parameters enter, (2) what condition it checks, (3) what state it changes or value it returns, and (4) why that matters to the player. Point to the exact lines as you speak.